

Progress Report: StateCraft Noise Layer (Phase 4)

Name: Omar Alshammari
ID: 200819560
University: KFUPM
Course: COE 619

Introduction (Phases 1-3 Summary)

Phase 1 established the project foundation: a multi-module Gradle layout, GraalVM native-image configuration, a CLI entry point, and initial tooling scaffolding. Phase 2 delivered the circuit ingest pipeline by adding a parser API plus JSON and OpenQASM 3 (subset) parsers, format detection, and end-to-end tests that run parsed circuits on the engine. Phase 3 completed the statevector MVP with SIMD-accelerated kernels, measurement sampling (counts and samples), and a small algorithm suite (e.g., Bell, GHZ, QFT) backed by unit and integration tests. These phases provided a stable engine and API surface to integrate noise in Phase 4.

Phase 4 Objectives

- Integrate a configurable noise model into the statevector engine.
- Apply Kraus-operator sampling after each unitary gate.
- Support deterministic noise trajectories via seeded RNG.
- Validate behavior with targeted unit tests.

Phase 4 Implementation Details

Noise model and channels

- `NoiseModel` supports three attachment modes: gate-type specific, per-qubit, and global-after-every-gate. A builder produces immutable models; the legacy fluent constructor remains for backward compatibility.
- `NoiseModel.channelsAfter(Operation)` aggregates gate, qubit, and global channels based on the operation's affected qubits.
- `ErrorChannel` provides factory methods for depolarizing, amplitude damping, phase flip, phase damping, thermal relaxation, and composite channels.
- `KrausDecomposition` validates operator dimensions and probability normalization and exposes `sampleOperator(SplittableRandom)` for Monte Carlo sampling.

SimulationRequest integration

- `SimulationRequest` now carries optional `NoiseModel` and optional `noiseSeed`, alongside the circuit, initial state, and measurement instruction.

- Request validation ensures qubit counts and measurement constraints are respected and that either a final state or measurement is requested.

Statevector engine integration

- `StatevectorEngine` instantiates a `SplittableRandom` for noise sampling when a model is provided.
- After each unitary operation, the engine calls `applyNoiseAfterOperation` and applies each channel in the list returned by the noise model.
- `applyErrorChannel` handles composite channels recursively, rejects multi-qubit channels, and checks qubit ranges.
- `applySingleQubitKraus` applies the Kraus matrix via the existing single-qubit kernel and then renormalizes the state.
- Measurements remain a terminal suffix; noise is not applied after measurements, and unitary gates after measurement are rejected.

Code Snippet (Engine Integration)

```
NoiseModel noiseModel = request.noiseModel().orElse(null);
boolean applyNoise = noiseModel != null && noiseModel.hasNoise();
SplittableRandom noiseRng = applyNoise
    ? (request.noiseSeed().isPresent()
        ? new SplittableRandom(request.noiseSeed().getAsLong())
        : new SplittableRandom())
    : null;

for (QuantumCircuit.Operation operation : circuit.operations()) {
    if (operation instanceof QuantumCircuit.Operation.SingleGateOperation single) {
        applySingle(state, single);
        if (applyNoise) {
            applyNoiseAfterOperation(state, operation, noiseModel, noiseRng, qubitCount);
        }
    }
    // other unitary operations omitted for brevity
}
```

Verification and Proof

The proof is encoded in tests that check deterministic sampling and expected operator selection. The excerpt below shows that a seeded run selects the same Kraus operator in both engine and test code:

```
SimulationRequest request = SimulationRequest.zeroState(circuit)
    .withNoiseModel(model)
    .withNoiseSeed(seed);

SimulationResult result = engine.simulate(request);
```

```

StateVector finalState = result.finalState().orElseThrow();

KrausDecomposition decomposition = channel.krausDecomposition();
int operatorIndex = decomposition.sampleOperator(new SplittableRandom(seed));
KrausOperator operator = decomposition.operators().get(operatorIndex);
double[] expected = applyOperator(new double[] {0.0, 0.0, 1.0, 0.0}, operator);

assertStateEquals(expected, finalState);

```

Sample outputs from local runs show results with and without noise across two cases:

```

Case 1: H on |0>
  Without noise: [0.7071067811865475, 0.0, 0.7071067811865475, 0.0]
  With phase-flip noise (p=1.0): [0.7071067811865475, 0.0, -0.7071067811865475, 0.0]
Case 2: X on |0>
  Without noise: [0.0, 0.0, 1.0, 0.0]
  With amplitude damping (gamma=0.4, seed=0): [1.0, 0.0, 0.0, 0.0]

```

Challenges and Resolutions

- Deterministic sampling required consistent RNG behavior between engine and tests; resolved by injecting `noiseSeed` and using `SplittableRandom` on both sides.
- Kraus application changes state norm; a renormalization step was added after each operator to prevent drift.
- Multi-qubit Kraus operators are not yet supported; explicit exceptions were added to fail fast.
- Noise application had to respect the measurement suffix constraint; noise is only applied to unitary operations.

Current Limitations

- Only single-qubit Kraus operators are supported.
- Idle-time noise is not modeled.
- Noise configuration is API-only; the CLI does not expose it.
- One noise trajectory is sampled per simulation; no trajectory averaging is provided yet.

Next Steps

1. Add CLI flags and configuration loaders for noise models.
2. Extend support to multi-qubit Kraus operators in the engine and channel definitions.
3. Introduce idle-time noise using circuit scheduling or explicit gate-time metadata.

4. Add trajectory averaging utilities to estimate noisy distributions more accurately.